



Surplus Signal Protocol

A minimal, federated coordination grammar for mutual provisioning

The SSP v0.2 specification — a semi-legible protocol that lets communities signal surplus and need without money, central databases, or identity surveillance.

By **Björn Kenneth Holmström**

with Claude, ChatGPT, DeepSeek, Mistral, Gemini and Grok

Global Governance Frameworks

Surplus Signal Protocol v0.2

A minimal, federated, semi-legible coordination grammar for mutual provisioning.

Preamble – Constitutional Principles

These are not laws. They are the spirit of the protocol. Implementations that violate them may still function, but they diverge from what SSP is meant to be.

1. **Local sovereignty is primary.** No signal compels action. A node may ignore or refuse any signal without justification.
 2. **No mandatory global identity.** Identifiers are pseudonymous by default. No universal human identity verification shall be required.
 3. **No central registry.** There is no single global database of signals. Federation is opt-in and limited to trust zones.
 4. **Decay by default.** All reputation, visibility, and appreciation metrics must decay over time (default ≤ 30 days).
 5. **Opacity is preserved by design.** Precise location and verified identity are disclosed only with explicit, time-limited consent.
 6. **Forkability is a right.** The protocol, its implementations, and even this preamble may be forked under permissive terms (CC0).
 7. **Hybrid neutrality.** The protocol does not favour or disfavour any `transaction.type` ; money and non-money flows coexist without hierarchy.
 8. **AI is advisory only.** Any automated matching, forecasting, or suggestion must be overridable by a human. No fully automated allocation is permitted.
 9. **No mandatory global metrics.** Success is measured locally. There is no enforced global key performance indicator.
 10. **Exit must be real.** A node can leave, delete its historical signals, and take its data (see §8).
-

1. Introduction

SSP is not an economy. It is a **sovereignty architecture**—a way for autonomous systems (bioregions, cooperatives, neighbourhoods, mutual aid groups) to coordinate flows of surplus and need without collapsing into domination, fragmentation, or extraction.

It is a **protocol, not a platform**. It prescribes no central database, no universal identity, and no permanent reputation. It is designed to be **semi-legible**: enough visibility to route resources, enough opacity to protect human relationships.

SSP starts from one observation: modern economies waste enormous amounts not because we lack resources, but because needs and capacities cannot **see** each other. This protocol is a thin grammar for making them visible—on terms that respect local autonomy.

2. General Signal Envelope

Every SSP signal carries a shared envelope.

Field	Type	Required	Description
schema_version	string	yes	"0.2"
signal_type	string	yes	One of the registered signal types
id	UUID	yes	Unique identifier generated by the issuing node
node_id	string	yes	Pseudonymous, stable per node; derived from a public key (e.g., Base58-encoded hash)
timestamp	ISO 8601	yes	Creation time in UTC
expires	ISO 8601	no	When the signal ceases to be valid; default 7 days
location	object	yes*	(for offers/requests) See §2.1
resource	object	yes*	(for offers/requests) What is being offered or requested
temporal	object	no	Time constraints beyond simple expiry
transaction	object	yes	Legal framing for local compliance
reputation	object	no	How reliability should be interpreted locally
disclosure	object	no	Graduated disclosure preferences
trust_generation	object	no	Bootstrapping trust in low-trust contexts
culture	object	no	Exchange norm declarations
metadata	object	no	Extensible key-value tags, notes
signature	string	yes	Digital signature over the signal's content (excluding signature)

2.1 Location Block

Preserves opacity while enabling logistics.

```
"location": {
  "coarse": "Upplands Väsby",           // mandatory, human-readable
  "precise": {"lat": 59.5, "long": 17.9}, // optional, null if not disclosed
  "pickup": true,
  "delivery": false
}
```

2.2 Resource Block

```
"resource": {
  "type": "food",           // core category (see §9)
  "subtype": "tomatoes",    // free-text or community subtype
  "quantity": 30,
  "unit": "kg",
  "quality": "organic",     // optional: "conventional", "surplus", "imperfect"
  "tags": ["vegan", "gluten_free"] // optional folksonomy
}
```

2.3 Temporal Block

```
"temporal": {
  "availability_start": "2026-05-09T08:00:00Z",
  "availability_end": "2026-05-11T18:00:00Z",
  "needed_by": null, // for requests
  "recurrence": { // optional
    "frequency": "weekly",
    "end_date": "2026-12-31",
    "occurrences": 10
  }
}
```

2.4 Transaction Block

Encodes the social-legal frame. All types are self-declared.

```
"transaction": {
  "type": "gift", // "gift", "barter", "mutual_credit", "non_profit", "hybrid"
  "legal_entity": "org_num_123456-1234", // optional
  "currency_accepted": null, // if hybrid, e.g. "SEK"
  "vat_applicable": false,
  "receipt_required": false,
  "jurisdiction": "SE" // self-declared
}
```

2.5 Reputation Block (optional)

Only included when a node wants to influence how its signals are treated locally. No global reputation is maintained.

```
"reputation": {
  "decay_rate": "14d", // default: halving every 14 days
  "dimensions": ["food_handling", "reliability"]
}
```

2.6 Disclosure Block (optional)

Controls graduated release of preciseness.

```
"disclosure": {
  "current_level": 0, // 0 = coarse only; 1 = precise location; 2 = verified identity
  "thresholds": {
    "level_1": 5, // after 5 successful fulfillments
    "level_2": 10
  },
  "consent_required": true,
  "consent_timeout": "24h"
}
```

2.7 Trust Generation Block (optional)

For low-trust environments.

```

"trust_generation": {
  "vouches": [
    {
      "from_node": "Base58voucher",
      "domain": "food",
      "weight": 0.8,
      "expires": "2026-08-01T00:00:00Z"
    }
  ],
  "collateral": {
    "type": "labor_pledge",
    "amount": 10,
    "unit": "hours",
    "description": "Community garden maintenance",
    "validated_by": null
  }
}

```

2.8 Culture Block (optional)

Declares local exchange norms to prevent friction.

```

"culture": {
  "exchange_norm": "gift_with_reciprocity_expected",
  "notes": "In our community, a return gift of equal value is customary but not enforced."
}

```

2.9 Metadata Block (optional)

```

"metadata": {
  "notes": "Harvested this morning. Bring your own containers.",
  "urgency": "medium"
}

```

3. Core Signal Types

3.1 Offer – Surplus or Capacity Declaration

An `offer` uses the full envelope. The `resource` and `location` blocks are mandatory. An offer may include `temporal` constraints and `recurrence`.

Example

```

{
  "schema_version": "0.2",
  "signal_type": "offer",
  "id": "d290f1ee-6c54-4b01-90e6-d701748f0851",
  "node_id": "12D3KooWQkG9RN4nQJ6...",
  "timestamp": "2026-05-09T08:00:00Z",
  "expires": "2026-05-11T14:00:00Z",

```

```

"location": {
  "coarse": "Upplands Väsby",
  "precise": null,
  "pickup": true,
  "delivery": false
},
"resource": {
  "type": "food",
  "subtype": "tomatoes",
  "quantity": 30,
  "unit": "kg",
  "quality": "organic",
  "tags": ["heirloom", "sauce_tomatoes"]
},
"transaction": {
  "type": "gift",
  "legal_entity": "REK0_Väsby_1"
},
"reputation": {
  "decay_rate": "14d",
  "dimensions": ["food_handling"]
},
"metadata": {
  "notes": "Harvested this morning. Please bring your own containers."
},
"signature": "0x4a8b2c..."
}

```

3.2 Request – Need Declaration

A `request` mirrors the offer structure. It may include a `buffer` field for partial fulfillment tolerance.

```

"buffer": {
  "min_quantity": 25,
  "max_quantity": 30
}

```

3.3 Fulfillment – Outcome Record

The feedback loop. A `fulfillment` references an offer and a request and reports the result. It is optional but strongly encouraged—it feeds local learning without creating permanent scores.

```

{
  "schema_version": "0.2",
  "signal_type": "fulfillment",
  "id": "uuid-fulfillment-1",
  "node_id": "Base58nodeA",
  "timestamp": "2026-05-09T14:30:00Z",
  "references": {
    "offer_id": "uuid-offer-123",
    "request_id": "uuid-request-456"
  },
}

```

```

"outcome": {
  "status": "completed",
  "quantity_fulfilled": 30,
  "unit": "kg",
  "reason_failure": null
},
"feedback": { // optional
  "rating": 5,
  "note": "Perfect condition!",
  "tags": ["on_time", "friendly"]
},
"signature": "...
}

```

3.4 Shortage Alert – System-Generated Advisory

Generated by a node's analytics to surface persistent gaps. It is purely advisory and meant to spark human conversation.

```

{
  "schema_version": "0.2",
  "signal_type": "shortage_alert",
  "id": "uuid-alert-789",
  "node_id": "Base58analyticsNode",
  "timestamp": "2026-05-09T12:00:00Z",
  "resource": { "type": "food", "subtype": "winter_greens" },
  "period": { "start": "2026-11-01", "end": "2026-03-31" },
  "deficit_rate": 0.4,
  "current_production_intents": 2,
  "suggested_target": 5,
  "convening_suggestion": "Consider a bioregional food assembly or REKO meeting.",
  "scenario": {
    "if": { "new_intents": 3, "capacity_sq_m": 150 },
    "then": { "estimated_deficit_rate": 0.05 }
  },
  "signature": "...
}

```

4. Production Coordination Extensions

These signals allow the protocol to move beyond surplus distribution into **adaptive production planning**—not central planning, but a conversational, commitment-based loop.

4.1 Production Intent

A node declares what it plans to produce.

```

{
  "signal_type": "production_intent",
  "resource": { "type": "food", "subtype": "kale" },

```

```

    "quantity": 100,
    "unit": "kg",
    "timeframe": "2026-Q3",
    "capacity_details": { "land_sq_m": 200, "method": "organic_no_till" }
  }

```

4.2 Capacity Offer

A node offers production assets (land, tools, labour).

```

{
  "signal_type": "capacity_offer",
  "resource": { "type": "space", "subtype": "garden_plot" },
  "quantity": 50,
  "unit": "sq_m",
  "availability": "2026-03 to 2026-10",
  "constraints": "prefer cooperative grower"
}

```

4.3 Collaboration Request

To find partners for a production goal.

```

{
  "signal_type": "collaboration_request",
  "goal": "cover_winter_greens_gap",
  "needed": { "type": "skill", "subtype": "farming" },
  "description": "Looking for 2 more growers in Väsby area."
}

```

Together, these three signals turn `shortage_alert`s into actionable local agendas. The protocol surfaces “who is willing” and “what is missing”; humans decide what to do.

5. Appreciation – Gratitude That Does Not Corrode

We deliberately avoid a points-based “Hearts” system. Legible appreciation, even with decay and locality, creates optimization pressure, social gaming, and metric-chasing.

Instead, appreciation is expressed in two ways:

5.1 Contribution to a Commons

The primary way to say “thank you” is to **offer something back to the network**—not to the individual, but to the shared infrastructure. An `offer` signal may include an optional `inspired_by` field referencing the original signal that prompted it.

```

"inspired_by": "uuid-of-original-offer"

```

Example: “*The school kitchen received such wonderful tomatoes that we’re offering 4 hours of deep-cleaning help to the community centre.*”

This keeps gratitude relational, generative, and un-metricised.

5.2 Ephemeral Appreciation Signal (optional, constrained)

If a direct “thank you” is desired, a node may emit an `appreciation` signal with the following mandatory constraints:

- **Ephemeral:** Must not be stored beyond 24 hours.
- **Non-scarce:** Unlimited in number.
- **Locally visible:** Not federated beyond the trust zone of origin.
- **Algorithmically ignored:** Bots, matching engines, and reputation modules must disregard it entirely.

```
{
  "signal_type": "appreciation",
  "reference_signal": "uuid-of-fulfillment",
  "expression": "gratitude",
  "note": "Your surplus made our school lunch wonderful!",
  "ephemeral": true
}
```

This is a digital smile. It carries no weight. It cannot be accumulated.

6. Cultural Mediation

When nodes from different exchange-norm backgrounds interact, friction is inevitable. The protocol provides an optional mediation scaffold.

6.1 Cultural Default Declaration (Node Metadata)

Nodes may declare their default norm in their node profile (not per signal):

```
"cultural_default": {
  "exchange_norm": "gift_with_reciprocity_expected",
  "description": "In our tradition, a return gift of equal value is customary but not enforced."
}
```

6.2 Mediation Signal

If a conflict arises, any involved node can issue:

```
{
  "signal_type": "cultural_mediation",
  "parties": ["nodeA", "nodeB"],
  "issue": "reciprocity_expectation_mismatch",
  "request": "seek_clarification",
  "mediator_suggestion": "some_bicultural_community_member"
}
```

6.3 Mediation Outcome

Mediation results are recorded as a non-binding signal:

```
{
  "signal_type": "mediation_outcome",
```

```
"mediation_id": "...",  
"resolution": "agreed_to_clarify_expectations",  
"description": "NodeB will explicitly state when a gift is pure gift."  
}
```

If mediation fails, the ultimate resolution is always **exit**—mute the other node, fork the trust zone, or leave.

7. Governance of the Protocol

Constitutional principles are necessary, but they must be complemented by explicit, minimal governance mechanics. No single entity owns SSP. Its evolution is guided by the following norms.

7.1 Schema Authority

- Changes are proposed publicly (e.g., GitHub repository under CC0).
- Implementations signal adoption by emitting the new `schema_version`.
- Backward-compatible extensions are encouraged; breaking changes require a new major version.
- Decisions follow rough consensus. No formal voting body exists.

7.2 Dispute Resolution

Disputes are handled in tiers:

1. **Local trust zone** resolves internally.
2. If cross-zone, cultural mediation is attempted.
3. If unresolved, any node may mute, unfederate, or fork.

7.3 Relay Operation

- A cross-zone relay is a dumb forwarder.
- It must publish a `relay_policy` signal:

```
{  
  "signal_type": "relay_policy",  
  "relay_type": "dumb_forwarder",  
  "filter_policy": "pass_through",  
  "logging_policy": "none"  
}
```

- Any node may operate a relay. No relay may claim special status.
- Local nodes monitor relay concentration. If >30% of signals in a zone pass through a single relay, an advisory is raised.

7.4 Fork Threshold

Forking is a first-class right. A node may signal a variant via `protocol_variant` in its metadata. Partial interop is maintained if the envelope and core signal types remain compatible.

8. Legal Shielding & Exit Mechanics

8.1 Node-Level Legal Metadata

Every node may include a `legal` block in its profile:

```
"legal": {
  "warrant_canary": "last_updated_2026-05-01",
  "data_retention_days": 7,
  "right_to_delete": true,
  "jurisdiction_acceptance": "this node operates under Swedish/EU law"
}
```

The warrant canary is a simple date; its absence (or staleness) hints at possible compelled disclosure, though it offers no technical guarantee.

8.2 Exit Protocol

- A node exits by broadcasting a `node_offline` signal with reason `"exit"`.
- Its historical signals are marked for deletion by peers (max 48-hour grace period, unless local law demands longer).
- The node may request a data export from peers (implementation-specific).
- No global blacklisting occurs.

9. Resource Type Vocabulary (Core)

To enable cross-node analytics while allowing local extension, the protocol defines a minimal controlled vocabulary for `resource.type`. Subtypes remain free-form.

Type	Description	Example Subtypes
<code>food</code>	Edible goods	tomatoes, kale, bread, honey
<code>tools</code>	Physical tools/equipment	drill, sewing_machine
<code>space</code>	Physical space	storage, workshop, garden_plot
<code>transport</code>	Movement of goods/people	bicycle, van, delivery_slot
<code>skill</code>	Human knowledge/labour	repair, teaching, childcare
<code>energy</code>	Power/heat	solar, wood, battery_storage
<code>care</code>	Health/elder/child care	meal_delivery, companionship
<code>digital</code>	Digital goods/services	software, 3d_printing, design

The type registry is maintained as a collaborative wiki. No central authority gatekeeps additions; local nodes may extend, but unknown types fall back to a generic parent.

10. Federation Topology

- **Within a trust zone:** Full peer-to-peer mesh via a gossip protocol, Matrix rooms, or Nostr relays. Each node maintains a local, ephemeral store of recent signals.
 - **Cross-zone routing:** Opt-in via relays (dumb forwarders). Discovery is not global.
 - **Global layer:** An eventual, highly aggregated commons registry (resource type wiki, contact points for trust zones). It does **not** contain live signals.
-

11. Implementation Guidance & Pilot Design

11.1 Human-Throughput First

All signals must be representable as plain text (JSON or compact YAML). A human can paste them into WhatsApp, Telegram, or a forum, and a parser can ingest them. A reference implementation (Matrix bot + local SQLite node) is provided, but not required.

11.2 AI-Assisted Matching (Optional, Advisory)

A local node may run a lightweight model (quantised LLM or constraint solver) that *suggests* matches, forecasts gluts, and simulates production coverage. All suggestions must be overridable. No model may act autonomously.

11.3 Pilot: Dual-Path Deployment

To verify the protocol's generality, two simultaneous pilots are recommended:

- **Path A (Trust-Rich, e.g., Swedish REKO ring):**
 - Start with bare-bones (offer/request/fulfillment) over WhatsApp/Matrix.
 - Introduce bot-assisted matching after human validation.
 - Test appreciation, production intent, and cultural defaults gradually.
 - Metric: coordination efficiency, user acceptance, legal compliance.
- **Path B (Low-Trust, e.g., migrant community hub):**
 - Start with hard-mode extensions: vouching, collateral, graduated disclosure, cultural mediation.
 - Use WhatsApp as transport; no dedicated hardware needed.
 - Goal: prove the protocol can bootstrap coordination where no prior trust exists.
 - Metric: number of successful fulfilments, qualitative trust emergence, conflict resolution patterns.

Comparison of both paths will determine whether SSP is a REKO upgrade or a true sovereignty architecture.

12. Licence & Contribution

SSP v0.2 is dedicated to the public domain under the [Creative Commons Zero \(CC0\)](#) licence. Implementations may use any licence. Contributions to the protocol are managed via rough consensus on the canonical repository.

This document is a living grammar, not a final textbook. It grows as communities test it, break it, and adapt it. The forest mycelium works because signals are local, chemical, and contextual. SSP aims for analogous sparseness in the human layer.